

Name : E. Rohan
Reg No : 192524482

SIMATS ENGINEERING ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)

Activity Tool : Analytical Problem Solving (High)
Assessment Tool Weightage: 35%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Apply relational algebra operations (σ , π , $\cup$, $-$, $\times$, $\bowtie$) on the given Employee and Department relations to retrieve required records.	BL3 – Apply	5
2	Write SQL DDL statements to create STUDENT(SID, Name, DOB, DeptID) and DEPARTMENT(DeptID, DeptName) tables with all appropriate constraints.	BL3 – Apply	5
3	Formulate SQL queries using GROUP BY, HAVING, and aggregate functions (COUNT, SUM, AVG, MAX, MIN) on a given Sales dataset.	BL3 – Apply	5
4	Write a correlated sub-query to find employees earning more than the average salary of their own department.	BL3 – Apply	5
5	Apply all types of JOIN operations (INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, CROSS) on Employee and Project tables.	BL3 – Apply	5
6	Create a view 'DeptSalaryView' that displays department name, total employees, and average salary. Write a query on this view.	BL6 – Create	5

7	Construct a relational algebra expression for the following: Find names of students who are enrolled in all courses offered.	BL6 – Create	5
8	Write SQL DML statements (INSERT, UPDATE, DELETE) with integrity constraint enforcement for a given scenario.	BL3 – Apply	5
9	Apply tuple relational calculus to express a query: Find all employees who work in the 'IT' department and earn more than 50000.	BL3 – Apply	5
10	Design a relational schema for an Airline Reservation System and write 5 SQL queries demonstrating joins, sub-queries, and views.	BL6 – Create	5

Code of Conduct:

I E. Rohan - 192524482 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

E. Rohan

RUBRICS FOR CALCULATION:

Numerical Problems					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding ; misses key elements	Misinterprets or unable to understand problem

Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with	Mostly correct steps with minor	Disorganized steps; multiple errors	No clear process
		correct approach throughout	mistakes		
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Q1.

Consider the following two relations:

EID	Name	Salary	DeptID
101	Arun	55000	D01
102	Bhavani	48000	D02
103	Chitra	72000	D01
104	Deepak	39000	D03

Table: Employee(EID, Name, Salary, DeptID)

DeptID	DeptName	Location
D01	Sales	Chennai
D02	HR	Bengaluru
D03	Finance	Chennai
D04	IT	Hyderabad

Table: Department(DeptID, DeptName, Location)

(a) Selection (σ) — employees earning above 50000

σ Salary > 50000 (Employee)

Result: {(101, Arun, 55000, D01), (103, Chitra, 72000, D01)}

(b) Projection (π) — only names and salaries

π Name, Salary (Employee)

Result: {(Arun,55000), (Bhavani,48000), (Chitra,72000), (Deepak,39000)}

(c) Union ($\cup$) — combine DeptIDs referenced by employees and DeptIDs that physically exist (both projections are union-compatible: single attribute, same domain)

π DeptID (Employee) $\cup$ π DeptID (Department)

Result: {D01, D02, D03, D04} — the set of every department ID that is either assigned to an employee or defined in Department.

(d) Set Difference ($-$) — departments that currently have no employees

π DeptID (Department) $-$ π DeptID (Employee)

Result: {D04} — the IT department exists but no employee is assigned to it.

(e) Cartesian Product ($\times$) — every Employee tuple paired with every Department tuple

Employee $\times$ Department

Produces $4 \times 4 = 16$ tuples, each of degree $4 + 3 = 7$ attributes (all combinations, including logically meaningless pairings where Employee.DeptID $\neq$ Department.DeptID). This is the base on which a Join is a filtered, meaningful subset.

(f) Join ($\bowtie$) — employees together with their department details

Employee ⋈ Employee.DeptID = Department.DeptID Department					
EID	Name	Salary	DeptID	DeptName	Location
101	Arun	55000	D01	Sales	Chennai
102	Bhavani	48000	D02	HR	Bengaluru
103	Chitra	72000	D01	Sales	Chennai
104	Deepak	39000	D03	Finance	Chennai

Note: D04 (IT) does not appear because the natural/theta join keeps only tuples with a matching DeptID on both sides — unlike the Cartesian product, no unmatched or meaningless combinations survive.

Combined query — “Names and salaries of Sales-department employees earning above 50000”

```
π Name, Salary ( σ Salary > 50000 ∧ DeptName='Sales' ( Employee ⋈ Employee.DeptID=Department.DeptID Department ) )
```

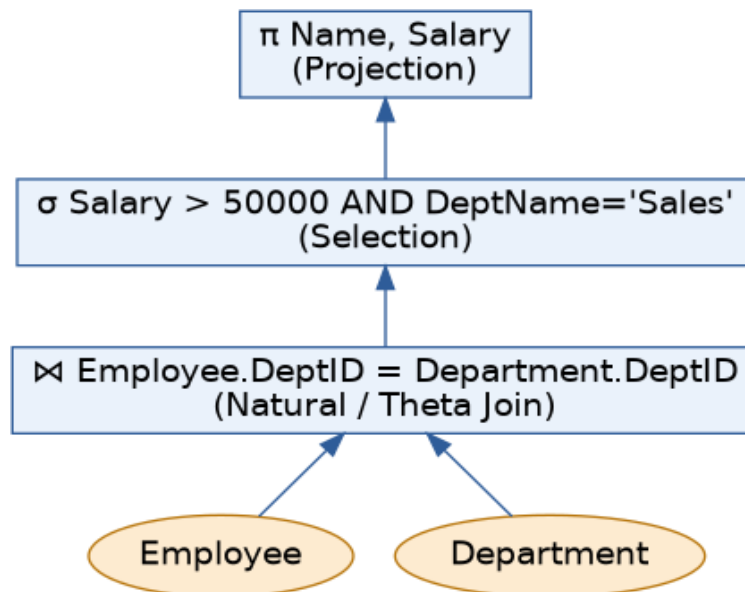


Fig. 1 – Relational algebra expression (operator) tree: operators are evaluated bottom-up, join first, then selection, then projection.

Result: {(Arun, 55000), (Chitra, 72000)} — both work in Sales and earn more than 50000.

Q2.

DEPARTMENT is created first because STUDENT will hold a foreign key that references it.

```
CREATE TABLE DEPARTMENT (  
    DeptID    VARCHAR(10)    PRIMARY KEY,  
    DeptName  VARCHAR(50)    NOT NULL UNIQUE  
);  
  
CREATE TABLE STUDENT (  
    SID       INT            PRIMARY KEY AUTO_INCREMENT,  
    Name      VARCHAR(50)    NOT NULL,  
    DOB       DATE           NOT NULL CHECK (DOB <= CURRENT_DATE),  
    DeptID    VARCHAR(10),  
    CONSTRAINT fk_student_dept  
        FOREIGN KEY (DeptID) REFERENCES DEPARTMENT(DeptID)  
        ON DELETE SET NULL  
        ON UPDATE CASCADE  
);
```

Constraints applied and why

- PRIMARY KEY on DeptID / SID — uniquely identifies each row, implicitly NOT NULL.
- UNIQUE on DeptName — prevents two departments from sharing the same name.
- NOT NULL on Name and DOB — every student must have a name and date of birth.
- CHECK (DOB <= CURRENT_DATE) — rejects a future date of birth (domain/data integrity).
- FOREIGN KEY (DeptID) — enforces referential integrity: a student can only be assigned to a department that already exists.
- ON DELETE SET NULL — if a department is removed, its students are not deleted, only ‘un-assigned’;
ON UPDATE CASCADE keeps DeptID in sync if it is ever changed.

Q3.

Sample dataset:

SaleID	Product	Region	Qty	Price
1	Laptop	South	2	55000
2	Mouse	South	10	500
3	Laptop	North	1	56000
4	Keyboard	South	5	1200
5	Mouse	North	8	500
6	Laptop	South	3	55000

Table: Sales(SaleID, Product, Region, Qty, Price)

(a) Total quantity sold in each region

```
SELECT Region, SUM(Qty) AS TotalQty
FROM Sales
GROUP BY Region;
```

(b) Number of distinct sale transactions per product, only for products sold more than once

```
SELECT Product, COUNT(*) AS NumSales
FROM Sales
GROUP BY Product
HAVING COUNT(*) > 1;
```

(c) Average price per product where the average exceeds 10000

```
SELECT Product, AVG(Price) AS AvgPrice
FROM Sales
GROUP BY Product
HAVING AVG(Price) > 10000;
```

(d) Highest and lowest priced item sold in each region

```
SELECT Region, MAX(Price) AS HighestPrice, MIN(Price) AS LowestPrice
FROM Sales
GROUP BY Region;
```

(e) Regions whose total revenue (Qty × Price) exceeds 150000

```
SELECT Region, SUM(Qty * Price) AS Revenue
FROM Sales
GROUP BY Region
HAVING SUM(Qty * Price) > 150000;
```

GROUP BY collapses rows sharing the same Region/Product into one summary row per group; the aggregate functions (SUM, COUNT, AVG, MAX, MIN) then compute over each group, while HAVING filters groups — unlike WHERE, which filters individual rows before grouping.

Q4.

```
SELECT E.EID, E.Name, E.Salary, E.DeptID
FROM Employee E
WHERE E.Salary > (
    SELECT AVG(E2.Salary)
    FROM Employee E2
    WHERE E2.DeptID = E.DeptID
);
```

Why this is “correlated”

The inner query references E.DeptID, a column from the outer query's row (alias E). This means the inner query cannot be evaluated once for the whole table — it must be re-executed once for every outer row, each time using that row's own DeptID to compute the average salary of just that department. This is what distinguishes a correlated sub-query from an ordinary (non-correlated) sub-query, which runs only once.

Trace using the Q1 Employee data

- Dept D01 average = $(55000 + 72000)/2 = 63500$ → Chitra (72000) qualifies, Arun (55000) does not.
- Dept D02 average = 48000 → Bhavani does not qualify (no one above the department's own single-row average).
- Dept D03 average = 39000 → Deepak does not qualify.

Final result: {Chitra}.

Q5.

EID	Name	DeptID
101	Arun	D01
102	Bhavani	D02
105	Esha	D01

Table: Employee(EID, Name, DeptID)

PID	PName	EID
P1	Website Revamp	101
P2	Payroll System	102
P3	Data Migration	999

Table: Project(PID, PName, EID) — note EID 999 has no matching employee, and employee 105 (Esha) has no project

(a) INNER JOIN — only employees that have a matching project

```
SELECT E.Name, P.PName
FROM Employee E
INNER JOIN Project P ON E.EID = P.EID;
-- Result: (Arun, Website Revamp), (Bhavani, Payroll System)
```

(b) LEFT OUTER JOIN — every employee, with NULLs where no project exists

```
SELECT E.Name, P.PName
FROM Employee E
LEFT JOIN Project P ON E.EID = P.EID;
-- Result: (Arun, Website Revamp), (Bhavani, Payroll System), (Esha, NULL)
```

(c) RIGHT OUTER JOIN — every project, with NULLs where no employee exists

```
SELECT E.Name, P.PName
FROM Employee E
RIGHT JOIN Project P ON E.EID = P.EID;
-- Result: (Arun, Website Revamp), (Bhavani, Payroll System), (NULL, Data Migration)
```

(d) FULL OUTER JOIN — all employees and all projects, matched where possible

```
-- Standard SQL:
SELECT E.Name, P.PName
FROM Employee E
FULL OUTER JOIN Project P ON E.EID = P.EID;

-- MySQL (no native FULL JOIN) -- simulate with UNION:
SELECT E.Name, P.PName FROM Employee E LEFT JOIN Project P ON E.EID = P.EID
UNION
SELECT E.Name, P.PName FROM Employee E RIGHT JOIN Project P ON E.EID = P.EID;
```

(e) CROSS JOIN — Cartesian product, every employee with every project

```
SELECT E.Name, P.PName
```

```
FROM Employee E
CROSS JOIN Project P;
-- Result: 3 employees x 3 projects = 9 rows
```

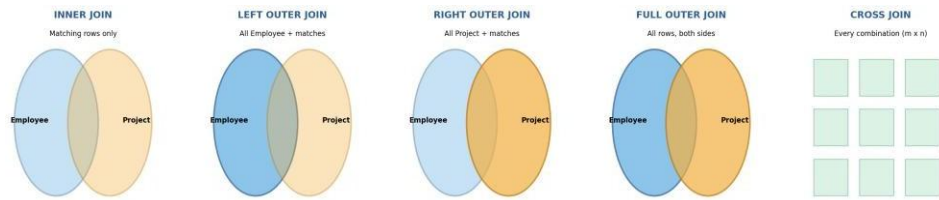


Fig. 2 – Visual comparison of the five JOIN types on Employee and Project.

Q6.

```
CREATE VIEW DeptSalaryView AS
SELECT D.DeptName,
       COUNT(E.EID) AS TotalEmployees,
       AVG(E.Salary) AS AvgSalary
FROM Department D
JOIN Employee E ON D.DeptID = E.DeptID
GROUP BY D.DeptName;
```

Query on the view — departments whose average salary exceeds 50000

```
SELECT DeptName, TotalEmployees, AvgSalary
FROM DeptSalaryView
WHERE AvgSalary > 50000
ORDER BY AvgSalary DESC;
```

A view stores no data of its own; each time it is queried, the server re-runs its defining SELECT. This lets a complex join-and-aggregate query on Department and Employee be reused as if it were a simple table, and it also hides the underlying base tables from users who only need the summary.

Q7.

Relations: Enrollment(SID, CID) and Course(CID), with Student(SID, Name).

This is a classic “for all” query, which relational algebra expresses with the Division ($\div$) operator: it returns the SID values in Enrollment that are paired with every single CID present in Course.

```
Result  $\leftarrow \pi \text{ SID, CID (Enrollment)} \div \pi \text{ CID (Course)}$ 
```

```
Final Answer  $\leftarrow \pi \text{ Name ( Student } \bowtie \text{ Student.SID=Result.SID Result )}$ 
```

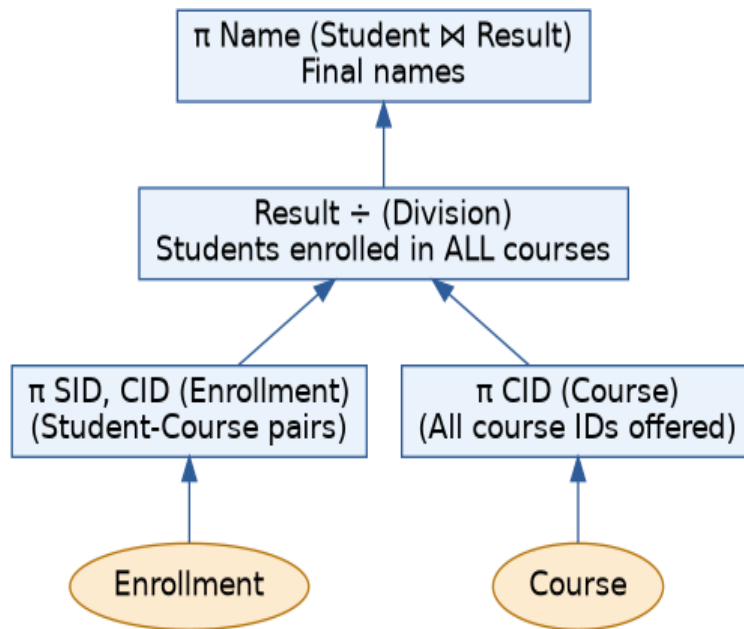


Fig. 3 – Expression tree for the division-based ‘enrolled in all courses’ query.

Equivalent expansion (Division defined using $-$ and $\times$, for completeness)

```
T1 =  $\pi \text{ SID, CID (Enrollment)}$   
T2 =  $\pi \text{ CID (Course)}$   
T3 = (  $\pi \text{ SID (T1)} \times \text{T2}$  ) - T1      -- (SID,CID) pairs that are 'missing'  
Result =  $\pi \text{ SID (T1)} - \pi \text{ SID (T3)}$   -- remove every SID that is missing at least one  
course
```

Interpretation: T3 lists every (student, course) combination the student should have taken but did not; subtracting the SIDs that appear in T3 leaves exactly the students who are missing nothing — i.e., enrolled in every course.

Q8.

Scenario: enforce constraints while inserting, updating, and deleting rows in STUDENT (from Q2), which has NOT NULL, CHECK, and FOREIGN KEY constraints.

(a) INSERT — valid row vs. constraint violation

```
-- Valid: DeptID 'D01' already exists in DEPARTMENT
INSERT INTO STUDENT (Name, DOB, DeptID)
VALUES ('Rahul', '2004-05-12', 'D01');

-- Violates FOREIGN KEY: 'D99' does not exist in DEPARTMENT -> rejected
INSERT INTO STUDENT (Name, DOB, DeptID)
VALUES ('Priya', '2004-01-20', 'D99');
-- Error: Cannot add or update a child row: a foreign key constraint fails
```

(b) UPDATE — valid change vs. CHECK violation

```
-- Valid: move a student to another existing department
UPDATE STUDENT SET DeptID = 'D02' WHERE SID = 1;

-- Violates CHECK (DOB <= CURRENT_DATE): a future date is rejected
UPDATE STUDENT SET DOB = '2030-01-01' WHERE SID = 1;
-- Error: Check constraint 'chk_dob' is violated
```

(c) DELETE — referential integrity on the parent table

```
-- Deleting a department: STUDENT.DeptID was declared ON DELETE SET NULL,
-- so this succeeds and every matching student's DeptID becomes NULL
DELETE FROM DEPARTMENT WHERE DeptID = 'D02';

-- Deleting a specific student row (no constraint blocks this,
-- since STUDENT is only ever the 'child' in this schema)
DELETE FROM STUDENT WHERE SID = 5;
```

In every case the DBMS checks NOT NULL, CHECK and FOREIGN KEY constraints before the row is written; a statement that would leave the database in an inconsistent state (an orphaned foreign key, an impossible date, a missing required value) is rejected and rolled back rather than partially applied.

Q9.

$$\{ t \mid t \in \text{Employee} \wedge t.\text{Dept} = \text{'IT'} \wedge t.\text{Salary} > 50000 \}$$

Reading the expression

- t is a tuple variable that ranges over every row of the Employee relation.
- $t \in \text{Employee}$ restricts t to tuples that actually exist in Employee (the range relation).
- $t.\text{Dept} = \text{'IT'}$ keeps only tuples whose Dept attribute equals 'IT'.
- $t.\text{Salary} > 50000$ further restricts to tuples whose Salary exceeds 50000; both conditions are joined with $\wedge$ (AND), so a tuple must satisfy both.

In words: “the set of all tuples t such that t is an Employee tuple, t works in the IT department, and t earns more than 50000.” If only certain columns were needed (say Name and Salary), the expression would be written as a set of projected attributes, e.g. $\{ t.\text{Name}, t.\text{Salary} \mid \dots \text{same condition} \dots \}$, which corresponds to $\pi_{\text{Name}, \text{Salary}} (\sigma_{\text{Dept}=\text{'IT'} \wedge \text{Salary}>50000} (\text{Employee}))$ in relational algebra — TRC and relational algebra are provably equivalent in expressive power.

Q10.

Entities and relationships

PASSENGER books a RESERVATION, each RESERVATION is for one FLIGHT and generates one PAYMENT. Each FLIGHT is operated by one AIRCRAFT and departs from / arrives at an AIRPORT.

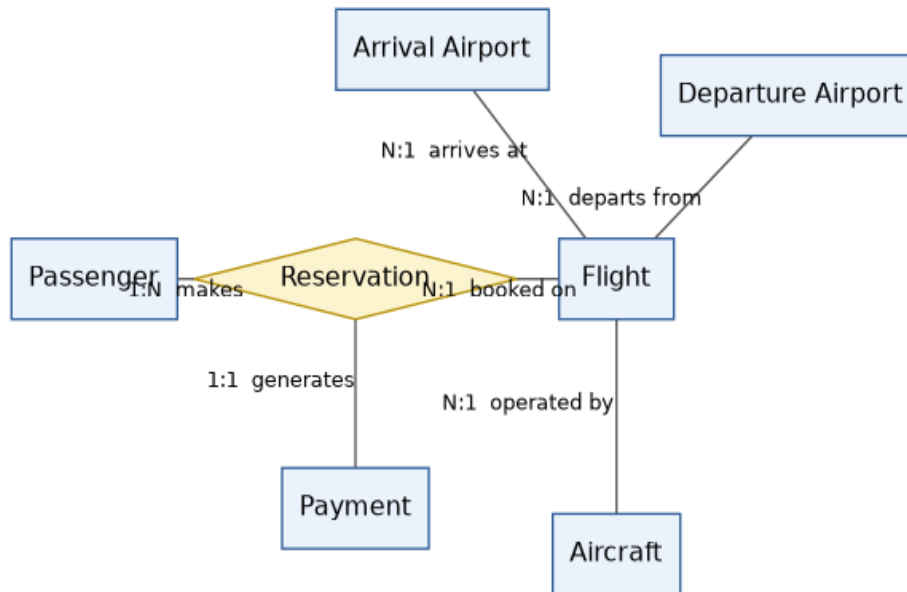


Fig. 4 – ER diagram of the Airline Reservation System.

Relational Schema

PASSENGER	(PassengerID PK, Name, Passport, Phone)
AIRPORT	(AirportCode PK, City, AirportName)
AIRCRAFT	(AircraftID PK, Model, Capacity)
FLIGHT	(FlightID PK, FlightNumber, DepAirport FK->AIRPORT, ArrAirport FK->AIRPORT, AircraftID FK->AIRCRAFT, DepartureTime, ArrivalTime)
RESERVATION	(ReservationID PK, PassengerID FK->PASSENGER, FlightID FK->FLIGHT, SeatNo, BookingDate, Status)
PAYMENT	(PaymentID PK, ReservationID FK->RESERVATION, Amount, PaymentDate, Mode)

Query 1 — JOIN: passenger names with the flights they booked

```
SELECT P.Name, F.FlightNumber, R.SeatNo, R.Status
FROM PASSENGER P
JOIN RESERVATION R ON P.PassengerID = R.PassengerID
JOIN FLIGHT F ON R.FlightID = F.FlightID;
```

Query 2 — Sub-query: passengers who have booked more than one flight

```
SELECT Name FROM PASSENGER
WHERE PassengerID IN (
    SELECT PassengerID FROM RESERVATION
    GROUP BY PassengerID
    HAVING COUNT(*) > 1)
```

```
);
```

Query 3 — View: seat occupancy per flight

```
CREATE VIEW FlightOccupancy AS
SELECT F.FlightNumber, AC.Capacity, COUNT(R.ReservationID) AS SeatsBooked
FROM FLIGHT F
JOIN AIRCRAFT AC ON F.AircraftID = AC.AircraftID
LEFT JOIN RESERVATION R ON F.FlightID = R.FlightID AND R.Status = 'CONFIRMED'
GROUP BY F.FlightNumber, AC.Capacity;

SELECT * FROM FlightOccupancy WHERE SeatsBooked >= Capacity; -- fully booked flights
```

Query 4 — Aggregate: total revenue collected per flight

```
SELECT F.FlightNumber, SUM(PM.Amount) AS TotalRevenue
FROM FLIGHT F
JOIN RESERVATION R ON F.FlightID = R.FlightID
JOIN PAYMENT PM ON R.ReservationID = PM.ReservationID
GROUP BY F.FlightNumber
ORDER BY TotalRevenue DESC;
```

Query 5 — Sub-query + LEFT JOIN: flights with zero reservations

```
SELECT F.FlightNumber, F.DepartureTime
FROM FLIGHT F
LEFT JOIN RESERVATION R ON F.FlightID = R.FlightID
WHERE R.ReservationID IS NULL;

-- Equivalent using a sub-query:
SELECT FlightNumber, DepartureTime FROM FLIGHT
WHERE FlightID NOT IN (SELECT FlightID FROM RESERVATION);
```